Passo 13: replace() para Corrigir Erros

Usamos o método `replace()` para trocar valores específicos na padronização de dados, garantindo consistência em colunas categóricas.

'cidade' (Original)	'cidade_padronizada'
'sp'	'São Paulo'
'Rio'	'Rio de Janeiro'
'BH'	'Belo Horizonte'
'SP'	'São Paulo'
'São Paulo'	'São Paulo'

```
df_corr = df.copy()
mapeamento = {
'sp': 'São Paulo', 'SP': 'São Paulo',
'rio': 'Rio de Janeiro', 'Rio': 'Rio...',
'BH': 'Belo Horizonte'
}
df_corr['cidade_padronizada'] = \
df_corr['cidade'].replace(mapeamento)
print(df_corr[['cidade',
'cidade_padronizada']])
```

Passo 14: O Problema dos Números como Texto

Quando uma coluna como **valor_compra** é importada como objeto (string), ficamos impossibilitados de realizar operações matemáticas diretas, como somas ou médias.

```
print(f"Tipo atual: {df['valor_compra'].dtype}")
df['valor_num'] = pd.to_numeric(df['valor_compra'], errors='coerce')
print(f"Novo tipo: {df['valor_num'].dtype}")
print(f"Soma total: R$ {df['valor_num'].sum():.2f}")
```

```
> Tipo atual: object
> Novo tipo: float64
> Soma total: R$ 1450.80
```

Passo 15: Limpando Formatos de Moeda (R$ 1.000,00)

O método `to_numeric` falha com strings de moedas complexas.

É obrigatório realizar a **limpeza completa** de caracteres especiais, delimitadores e espaços em branco antes de tentar a conversão.

> ⚠ **Por que falha?**
>
> Símbolos como **R$**, delimitadores de milhar (pontos) e vírgulas decimais impedem a análise do interpretador e resultam em `ValueError`.

```
valores_br = ['R$ 150,00', 'R$ 200,50']
s = pd.Series(valores_br)
# Limpeza em cadeia (Chaining)
s_limpo = (s.str.replace('R$', '', regex=False)
.str.strip()
.str.replace('.', '', regex=False)
.str.replace(',', '.'))
s_num = pd.to_numeric(s_limpo)
```

```
>>> print(s_num)
0 150.0
1 200.5
dtype: float64
```

Passo 16: to_datetime() - O Salva-Vidas

Datas salvas como **string** impedem a realização de filtros temporais e agrupamentos automáticos por mês ou ano.

A conversão para o tipo nativo **datetime64** é fundamental para habilitar análises cronológicas robustas no Pandas.

SAÍDA DO CONSOLE (TERMINAL)

```
> Tipo atual: object
> Novo tipo: datetime64[ns]

[data_compra, data_dt]:
0 "2026-08-13" ⟶ 2026-08-13
1 "14/08/2026" ⟶ 2026-08-14
```

```python
# 1. Verificar tipo original da coluna
print(f"Tipo atual: {df['data_compra'].dtype}")


# 2. Conversão inteligente to_datetime
df['data_dt'] = pd.to_datetime(
df['data_compra'],
format='mixed',
dayfirst=True
)


# 3. Confirmar tipo datetime64
print(f"Novo tipo: {df['data_dt'].dtype}")
print(df[['data_compra', 'data_dt']])
```

Passo 17: Acessor .dt para Datas

Uma vez que a coluna foi convertida para datetime, o acessor `.dt` permite extrair componentes específicos como ano, mês, dia e dia da semana de forma extremamente simples e direta.

```
df['ano'] = df['data_dt'].dt.year

df['mes'] = df['data_dt'].dt.month

df['dia_semana'] =
df['data_dt'].dt.day_name()

print(df['data_compra', 'ano', 'mes',
'dia_semana'])
```

data_dt	ano	mes	dia_semana
2026-08-13	2026	8	Thursday
2026-09-01	2026	9	Tuesday
2026-12-25	2026	12	Friday
2027-01-01	2027	1	Friday
2027-05-01	2027	5	Saturday

Passo 18: rename() para Clareza

Nomes de colunas confusos ou inadequados dificultam a leitura, o compartilhamento e a manutenção do pipeline de dados por outros membros da equipe.

```
df_ren = df.rename(columns={
    'valor_num': 'valor_compra_reais',
    'data_dt': 'data_formatada'
})
print(df_ren.columns)
```

```
# Output do terminal:
Index(['valor_compra_reais', 'data_formatada'], dtype='object')
```

Passo 19: Organizando o DataFrame Final

Reordenar colunas de forma lógica e sequencial melhora significativamente a usabilidade e a clareza visual do conjunto de dados final para a etapa de análise.

reordenar_colunas.py

```
colunas_finais = [
'id_pedido', 'cliente', 'idade',
'cidade_padronizada',
'valor_compra_reais',
'data_formatada'
]
df_final = df_ren[colunas_finais]
print(df_final)
```

Visualização de Saída: df_final

id_pedido	cliente	idade	cidade_padronizada	valor_compra	data_format
1001	Ana Silva	34	São Paulo	150.50	2026-08-10
1002	Bruno M.	28	Rio de Janeiro	89.90	2026-08-11
1003	Carlos Lima	45	Belo Horizonte	250.00	2026-08-11
1004	Daniela S.	22	Curitiba	120.00	2026-08-12
... [outras linhas de dados ocultadas] ...					

Cheat Sheet: Strings e Tipos

Resumo dos principais métodos de limpeza de strings e conversão no Pandas:

Limpeza de Strings

`df.drop_duplicates()`

Remove linhas duplicadas do dataset.

`df['col'].str.lower()`

Converte todo o texto para letras minúsculas.

`df['col'].str.strip()`

Remove espaços em branco nas bordas da string.

`df['col'].replace({})`

Substitui valores com base em um dicionário.

Conversão de Tipos & Datas

`pd.to_numeric()`

Converte a série para um tipo numérico.

`pd.to_datetime()`

Converte a série para o formato de data e hora.

`df['col'].dt.year`

Extrai a informação do ano de uma coluna datetime.

Desafio Prático

O Dataset "Imóveis Sujos"

Cenário de Negócios

Uma imobiliária precisa analisar preços de aluguel, mas a planilha enviada contém múltiplos erros estruturais e de digitação.

Sua Missão

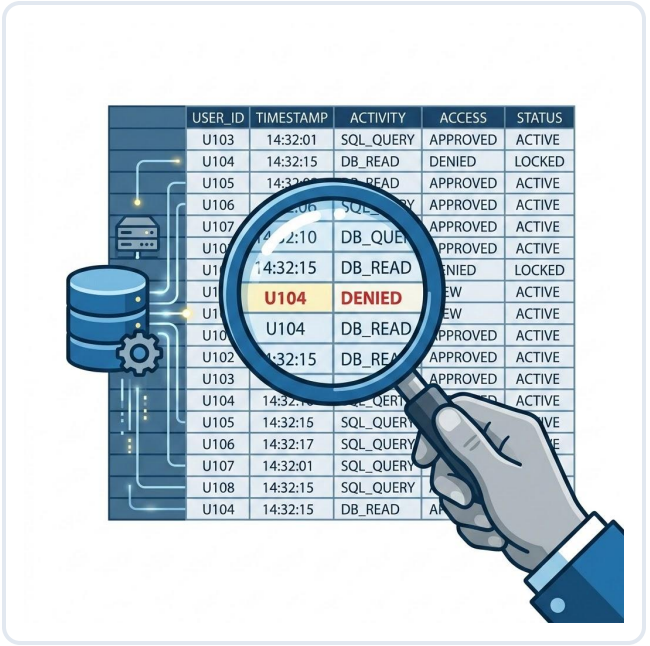
Limpar o dataset para habilitar análises confiáveis e salvar o DataFrame limpo resultante em formato CSV.

Tempo Limite

20 minutos

Formato de Saída

arquivo .csv



O que o Estagiário Enviou...

```python
# Problema estrutural "Imóveis Sujos"
dados_desafio = {
'id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
'bairro': ['Centro', 'centro ', 'Centro', ...],
'quartos': [2, 2, 'dois', 3, 3, 1, 1, 2, 3, 1],
'area_m2': ['100', '120', np.nan, '150', ...],
'aluguel': ['R$ 2.000,00', 'R$ 2.200,00', ...],
'condominio': [500, 500, 450, np.nan, 800, ...]
}
df_desafio = pd.DataFrame(dados_desafio)
print(df_desafio)
```

Visualização do DataFrame DADOS SUJOS

bairro	quartos	area_m2	aluguel	condo
Centro	2	"100"	R$ 2.000	500
centro	2	"120"	R$ 2.200	500
Centro	"dois"	NaN	R$ 1.800	450
Jardins	3	"150"	R$ 3.500	NaN
jardins	3	"150"	R$ 3.500	800

Problemas Críticos Identificados:
Inconsistência de texto/caixa, strings misturadas com números, e valores nulos (NaN) ocultos.

Passo 1: Diagnóstico

⏱️ Tempo sugerido: 5 min

Ação: Executar os métodos do Pandas no terminal para auditar a estrutura geral e mapear nulos no dataset.

audit_dataset.py

```
import pandas as pd
df_desafio =
pd.read_csv("dataset.csv")

# 1. Informações de tipo e contagem
df_desafio.info()

# 2. Totalizar valores nulos por
coluna
df_desafio.isnull().sum()
```

Problemas Encontrados:

⚠️ **1.** Bairros com diferenças de caixa e espaços adicionais.

⚠️ **2.** Coluna de quartos contendo texto literal (`'dois'`).

⚠️ **3.** Área com valores `NaN` e tipo incorreto (string).

⚠️ **4.** Aluguel formatado como moeda brasileira (string).

⚠️ **5.** Condomínio contendo dados faltantes (`NaN`).

⚠️ **6.** Presença de linhas duplicadas no conjunto.

Passo 2: Limpar Coluna 'bairro'

Ação de Limpeza

Padronizar os nomes dos bairros utilizando conversão de caixa, remoção de espaços em branco e capitalização adequada.

Conversão para caixa baixa

Remoção de espaços (strip)

Capitalização correta

```
# Padronizar strings
df_desafio['bairro'] =
df_desafio['bairro'].str.lower().str.strip()
df_desafio['bairro'] =
df_desafio['bairro'].str.capitalize()

print(df_desafio['bairro'].unique())
```

```
>_ stdout (array padronizado)
['Centro', 'Jardins', 'Vila mariana']
✔ Limpeza concluída com sucesso!
```

Passo 3: Corrigir Texto em Coluna Numérica

- Corrigir inconsistências de tipo na coluna `'quartos'` mapeando strings textuais literais para números.
- Converter a série para o tipo numérico adequado, forçando erros residuais como valores nulos.
- Preencher valores ausentes ou nulos com `0` e converter explicitamente a coluna para o tipo `int`.

limpeza.py

```
# 1. Mapear 'dois' para 2
df_desafio['quartos'] = df_desafio['quartos'].replace('dois', 2)

# 2. Forçar coerção numérica
df_desafio['quartos'] = pd.to_numeric(df_desafio['quartos'], errors='coerce')

# 3. Tratar NaNs e converter para inteiro
df_desafio['quartos'] = df_desafio['quartos'].fillna(0).astype(int)
```

CONSOLE OUTPUT

```
>>> print(df_desafio['quartos'].head())
0 3
1 2
2 1
3 0
4 4
Name: quartos, dtype: int64
✓ Tipo de dados atualizado
```

Passo 4: Lidar com NaN e Conversão

Ação: Imputação de Dados

Converter a coluna **`'area_m2'`** para o tipo numérico e imputar os valores nulos utilizando a **mediana**, evitando a perda de dados importantes.

ID 104:**NaN** → ID 104:**120.0 (Mediana)**

A conversão com **`errors='coerce'`** transforma valores inválidos em **NaN**, que são em seguida preenchidos de forma segura com o valor central da distribuição.

imputacao_mediana.py

```
# 1. Converter coluna para numérico
df_desafio['area_m2'] = pd.to_numeric(
df_desafio['area_m2'], errors='coerce' )

# 2. Calcular a mediana
mediana_area = df_desafio['area_m2'].median()

# 3. Imputar os valores nulos
df_desafio['area_m2'] =
df_desafio['area_m2'].fillna( mediana_area )
```

Passo 5: O Desafio da Moeda

Ação: Remover os símbolos de moeda brasileira, retirar espaços em branco e adequar os delimitadores numéricos para viabilizar operações aritméticas.

A conversão de strings de moeda (R$) para float exige o tratamento sequencial dos caracteres.

aluguel (Original)	aluguel_num (Float)
"R$ 1.500,00"	1500.0
"R$ 850,50"	850.5
"R$ 2.300,00"	2300.0
dtype: object	dtype: float64

limpeza_dados.py

```python
df_desafio['aluguel_limpo'] = (
  df_desafio['aluguel']
  .str.replace('R$', '', regex=False)
  .str.strip()
  .str.replace('.', '', regex=False)
  .str.replace(',', '.')
)
df_desafio['aluguel_num'] = (
  pd.to_numeric(df_desafio['aluguel_limpo'])
)
```

Passo 6: Preencher Condomínio

Tratar os valores nulos presentes na coluna **'condominio'**, preenchendo as ocorrências de NaN com a mediana calculada da própria coluna.

python

```python
# Calcular a mediana da coluna
mediana_cond = df_desafio['condominio'].median()

# Preencher as ocorrências nulas
df_desafio['condominio'] =
df_desafio['condominio'].fillna(mediana_cond)
```

Por que usar a Mediana?

A mediana é uma medida de tendência central robusta a outliers (valores discrepantes extremos), sendo altamente indicada para dados imobiliários e de custo.

Console / Verificação Estatística

```
>>> df_desafio['condominio'].describe()
count 1500.000000
mean 422.500000
50% 380.000000 # Mediana aplicada
max 1500.000000
>>> df_desafio['condominio'].isnull().sum()
0 # Sucesso: Sem valores nulos!
```

0

Valores Nulos Restantes em 'condominio'

Passo 7: Deduplicação Final

Ação: Auditar o DataFrame em busca de linhas perfeitamente idênticas e realizar a exclusão dessas duplicatas do conjunto de dados.

```
print(f"Duplicatas antes: {df_desafio.duplicated().sum()}")
df_desafio = df_desafio.drop_duplicates()
print(f"Duplicatas depois: {df_desafio.duplicated().sum()}")
```

```
>_ CONSOLE OUTPUT
Duplicatas antes: 142
Duplicatas depois: 0
```

Passo 8: O Dataset está Limpo?

Ação: Executar diagnósticos finais utilizando os métodos **`info()`** e **`describe()`** para validar se todas as transformações de tipo e imputações estão coerentes.

```
print(df_desafio.info())
print(df_desafio.describe())
```

> Saída do Terminal (Diagnóstico dos dtypes e valores nulos)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entradas, 0 a 999
Data columns (total 3 colunas):
 #   Coluna          Não-Nulos     Dtype
---  ------          ---------     -----
 0   id_cliente      1000 non-null  int64
 1   score_vendas    1000 non-null  int64     [✓ Tipo Correto]
 2   metrica_lucro   1000 non-null  float64   [✓ 0% Nulos / Imputado]
dtypes: float64(1), int64(2)
uso da memória: 23.5 KB
```

Passo 9: Exportar para CSV

Ação: Salvar o DataFrame completamente higienizado em um formato CSV seguro para uso posterior na etapa de análise exploratória de dados.

export_csv.py

```
df_desafio.to_csv('imoveis_limpos.csv', index=False)
print("✅ Dataset limpo salvo!")
```

Exportação Concluída

imoveis_limpos.csv

Console de Saída

```
$ python export_csv.py
✅ Dataset limpo salvo!
```

Hora do Show:
Como vocês resolveram?

DEBATE EM GRUPO (5 MIN)

Discussão das diferentes abordagens adotadas para a resolução do desafio.

Tópicos de discussão:

- Remoção de linhas nulas vs. imputação de mediana.
- Estratégias para o tratamento do padrão monetário brasileiro.
- Como as decisões de negócio moldam as regras de limpeza aplicadas aos dados.



Armadilhas: O que não fazer

⚠ **Principais erros durante a higienização de dados:**

- **1. Preencher tudo com 0 cegamente:** causa distorção severa na média.
- **2. Apagar todas as linhas com NaN de forma indiscriminada:** reduz drasticamente o volume do dataset.
- **3. Esquecer de tratar strings antes de agrupamentos:** diferenciar 'SP' de 'sp' gera duplicidade de chaves.
- **4. Deixar de verificar o dtype resultante pós-tratamento.**
- **5. Modificar dados no arquivo de origem diretamente:** sem reter backups adequados.

Checklist do Cientista de Dados

Práticas recomendadas no cotidiano de dados

Cópia de Segurança
Realizar sempre uma cópia do DataFrame via `df.copy()` antes de iniciar mutações.

Documentação das Alterações
Registrar documentação clara sobre modificações efetuadas diretamente no código (comentários estruturados).

Auditoria Estatística
Auditar alterações estruturais e de tipos com `df.info()` e `df.describe()` após cada modificação.

Validação e Plausibilidade
Verificar a plausibilidade física e lógica dos dados (ex: impedir idades negativas ou valores impossíveis).

Exportação Segura
Gravar a exportação higienizada em arquivo separado para preservar a fonte original (ex: `_limpo.csv`).

Dica Pro: Criando Funções de Limpeza

Automação em ETL

Em processos repetitivos de **ETL mensal**, empacotar as etapas dentro de funções estruturadas garante:

- **Eficiência:** Evita a reescrita de código redundante a cada ciclo.
- **Controle:** Centraliza as regras de negócio em um único ponto lógico.
- **Reproducibilidade:** Garante que novos conjuntos de dados sofram exatamente os mesmos tratamentos.

etl_pipeline.py

```python
def limpar_imoveis(df):
df = df.copy()
df['bairro'] =
df['bairro'].str.lower().str.strip()
# ... aplicar demais passos ...
return df

# Execução mensal automatizada:
df_novo_mes =
pd.read_csv('imoveis_outubro.csv')
df_limpo = limpar_imoveis(df_novo_mes)
```

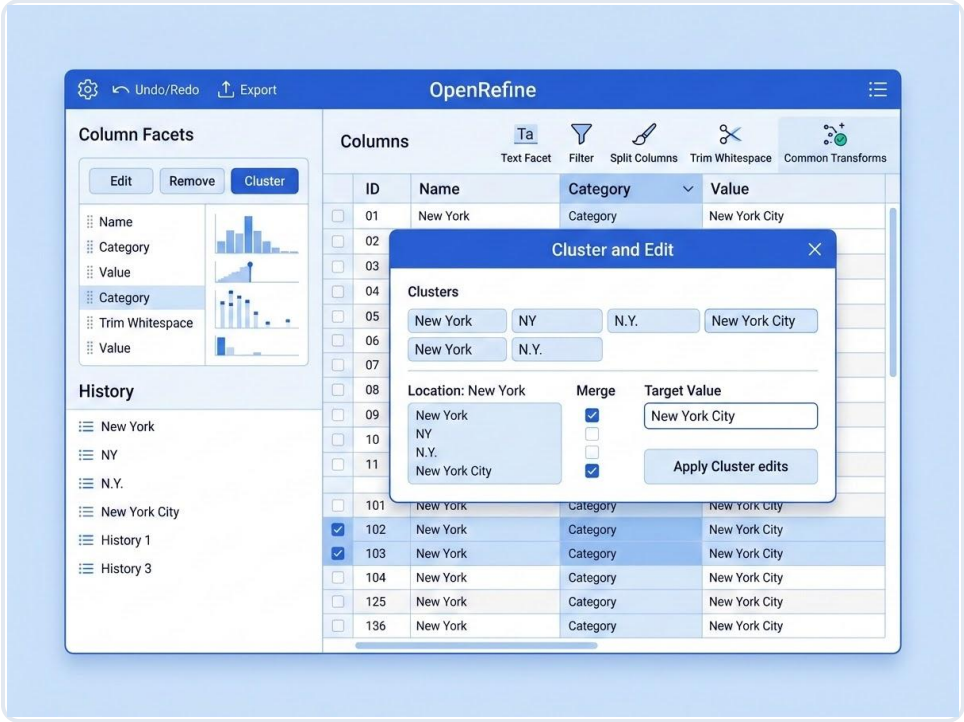
Além do Pandas: OpenRefine

Introdução ao OpenRefine

Uma ferramenta gráfica e interativa excelente para identificar erros textuais em massa, mesclar grafias semelhantes e tratar dados muito desorganizados antes mesmo do processamento no script Python.

Acesse online:

`openrefine.org`



O que Conquistamos Hoje?

Conquistado Hoje

Domínio de conceitos essenciais de limpeza:

- Tratamento estruturado de nulos (`dropna`, `fillna`, `ffill`).
- Remoção de redundâncias de dados e duplicatas.
- Transformações e padronização de campos de texto (`str.lower`, `strip`, `replace`).
- Conversões confiáveis para tipos numéricos e temporais.

Próxima Etapa

Próximos passos (Parte 2):

Normalização, engenharia de escala, codificação categórica e detecção avançada de anomalias (outliers).

O que Vem na Aula 6?

Introdução às técnicas de estruturação de dados avançada.

Conteúdo previsto:

- Identificação estrutural e tratamento de valores fora do padrão (outliers).
- Escalonamento e padronização (técnicas de Z-score e escala 0 a 1).
- Técnicas de codificação categórica (One-Hot Encoding, Label Encoding) para adequar dados para algoritmos de Machine Learning.

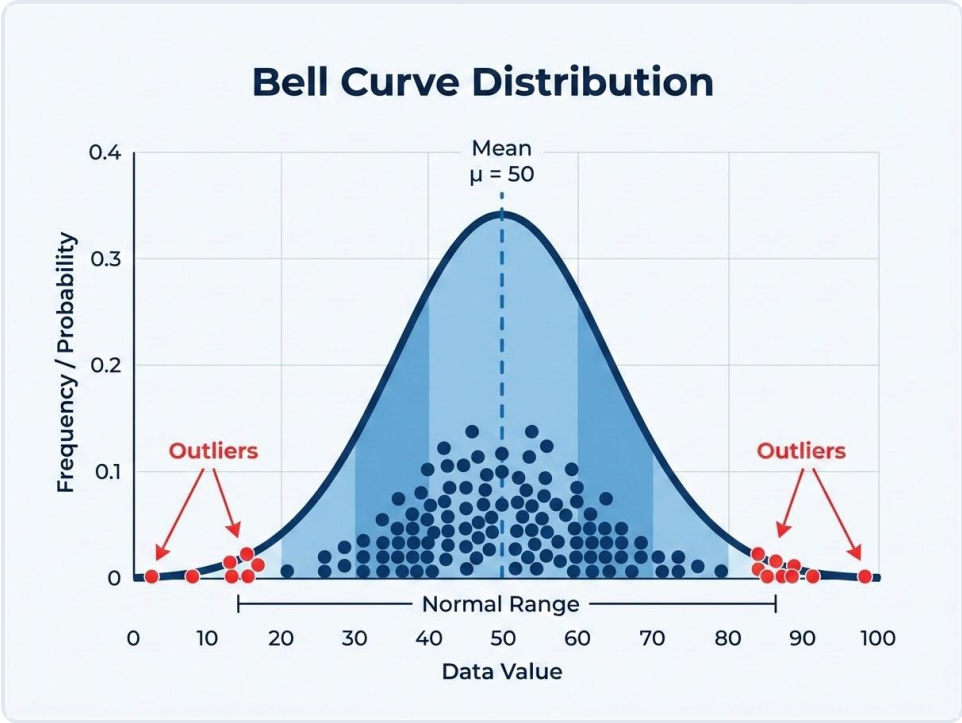


Gráfico de distribuição de dados ilustrando o conceito de outliers e preparação matemática para ML.

Continue Praticando

Referências Acadêmicas e Tutoriais para aprofundamento:

Pandas User Guide `Doc Oficial`

Foco na seção de dados nulos e métodos textuais para tratamento de DataFrames.

Artigos Especializados `Leitura`

"Data Cleaning in Python" **(Real Python)** e "How to Handle Missing Data" **(Towards Data Science)**.

Conteúdos Audiovisuais `Vídeo`

Tutoriais de qualidade no canal Keith Galli e canais de referência nacional de Python.

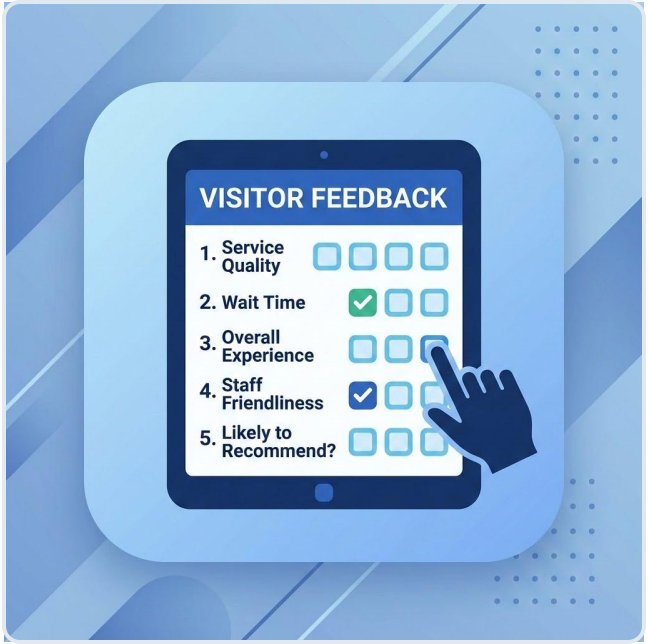
Prática Aplicada `Prática`

Resolver desafios práticos com bases de dados desestruturadas na plataforma Kaggle.

Sua Opinião Importa

Formulário de feedback (3 minutos):

- Avaliar o nível de dificuldade do material apresentado.
- Opinar sobre o ritmo e a divisão entre a teoria e a prática de dados.
- Medir a segurança individual ao assumir projetos de higienização de dados reais.
- Espaço para o envio de dúvidas e sugestões.



Preparação para Aula 6

Tarefas extracurriculares:

1. **1.** Revisar o notebook consolidado na aula de hoje.
2. **2.** Higienizar a base de dados coletada por você na Aula 4 aplicando o pipeline criado.
3. **3.** Trazer o notebook atualizado e o arquivo _limpo.csv para uso imediato no próximo encontro.
4. **4.** Realizar leitura preliminar sobre Outliers e algoritmos que necessitam de normalização de dados.

Parabéns! Seus Dados Estão Brilhando!

MENSAGEM DE ENCERRAMENTO

- **Um modelo sofisticado de ML falha miseravelmente** se for treinado com dados sujos e inadequados.
- **O sucesso na inteligência artificial inicia-se** na meticulosa engenharia de dados.
- **Agora você detém o poder** de converter inconsistências brutas em conhecimento confiável.